

# Voice Agent Platform — Engineering & Technical Manual

System Architecture · Audio Pipelines · Retell AI Webhook Engine · SQLite Receiver · Security

## 1. High-Level Architecture & Audio Processing Pipeline

The platform delivers sub-second conversational AI over public switched telephone networks (PSTN) and WebRTC. Audio ingestion supports two parallel carrier architectures: **Twilio Media Streams** (RFC 6455 bidirectional WebSocket carrying 8kHz G.711 mu-law audio) and **LiveKit SIP Trunks** (SIP signaling with Twirp RPC control and WebRTC media rooms).

| Pipeline Stage        | Technology / Provider              | Latency Target   | Operational Function                                            |
|-----------------------|------------------------------------|------------------|-----------------------------------------------------------------|
| 1. Audio Ingestion    | Twilio Media Streams / LiveKit SIP | < 20 ms          | Bidirectional 8kHz mu-law audio chunking (20ms frames)          |
| 2. Streaming STT      | Deepgram Nova-3 (WebSocket)        | 100 - 180 ms     | Real-time interim transcription with automatic endpointing      |
| 3. LLM Inference      | Gemini 2.5 Flash / GPT-4o-mini     | 180 - 320 ms     | Prompt evaluation, tool calling, and streaming token generation |
| 4. Voice Synthesis    | Deepgram Aura / Cartesia Sonic     | 90 - 150 ms      | Direct 8kHz mu-law audio synthesis streamed to phone caller     |
| 5. Post-Call Analysis | Gemini 2.5 Flash Pipeline Worker   | 20 - 35 ms async | Summary, sentiment polarity, and schema entity extraction       |
| 6. Event Dispatch     | Webhook Dispatcher (HMAC-SHA256)   | < 15 ms fan-out  | Signed Retell AI delivery to external receiver endpoints        |

## 2. Concurrent Multi-Caller Architecture (Same DID)

A critical engineering requirement verified in this system is **simultaneous caller support**. When 10, 50, or 100+ callers dial the exact same phone number configured on an agent, each incoming SIP INVITE or Twilio webhook allocates an independent, isolated session (TwilioMediaStreamSession or LiveKit Room) with its own Deepgram STT stream, LLM conversation history, and audio mixdown buffer. Sessions share zero memory state.

## 3. Webhook Delivery Engine (HMAC-SHA256 & DLQ)

The webhook delivery subsystem (agent/webhook\_dispatcher.py) ensures reliable event fan-out with external systems:

- **HMAC-SHA256 Signatures:** Header X-Signature-256 and X-Retell-Signature computed over <timestamp>.<body>.
- **Anti-Replay Window:** 300-second timestamp drift tolerance + in-memory Delivery ID cache.
- **Exponential Backoff Retries:** 1s, 2s, 4s, 8s with ±25% randomized jitter to avoid thundering herds.
- **Dead-Letter Queue (DLQ):** Unreachable endpoints are parked for review and one-click replay.

Call Desk / Webhooks & Data Hub

Refresh DataAPI Keys & Providers

Call Desk

AI Assistant Voice

Overview

Calls

Agents

API Keys & Providers

Webhooks & Data

User Guide

admin

admin@voiceagent.local

Primary Admin

PRODUCT ADMIN

Open profile

Logout

REAL-TIME INTEGRATIONS • LOCAL ENVIRONMENT

Webhooks & Event Dispatcher

New Endpoint

TOTAL DELIVERIES

1

1 total attempts

SUCCESS RATE

100%

1 acknowledged

DEAD-LETTER QUEUE

0

Permanently parked events

AVERAGE LATENCY

55.1 ms

Round-trip execution

ACTIVE ENDPOINTS

0

0 registered

Endpoints & Subscriptions

Live Delivery Logs

Dead-Letter Queue

Webhook Simulator

HMAC-SHA256 Sandbox

AI Data Extraction

Signature Verification Sandbox

HMAC-SHA256 testbench

SIGNING SECRET (WHSEC\_...)

whsec\_123456...

TIMESTAMP (X-WEBHOOK-TIMESTAMP)

1718800001

EXPECTED SIGNATURE (X-SIGNATURE-256)

sha256=...

RAW REQUEST BODY (JSON)

{  
 "event": "call\_completed",  
 "call\_id": "test-call-123"  
}

Test Signature Match

Receiver Implementation Examples

Node.js • Python • Go

PYTHON (FASTAPI/FLASK)

import hmac, hashlib, time  
  
def verify\_webhook(secret: str, body: str, timestamp: str, signature: str):  
 if abs(time.time() - int(timestamp)) > 300:  
 return False # Reply prevention  
 signed = f"[timestamp].[body]".encode("utf-8")  
 expected = "sha256=" + hmac.new(secret.encode(), signed, hashlib.sha256).hexdigest()  
 return hmac.compare\_digest(expected, signature)  
  
NODE.JS (EXPRESS)  
  
const crypto = require("crypto");  
  
function verifyWebhook(secret, bodyStr, timestamp, signature) {  
 if (Math.abs(Date.now() / 1000 - Number(timestamp)) > 300) return false;  
 const hmac = crypto.createHmac("sha256", secret).update(`\${timestamp}.\${bodyStr}`).digest("hex");  
 return crypto.timingSafeEqual(Buffer.from(hmac), Buffer.from(signature));  
}

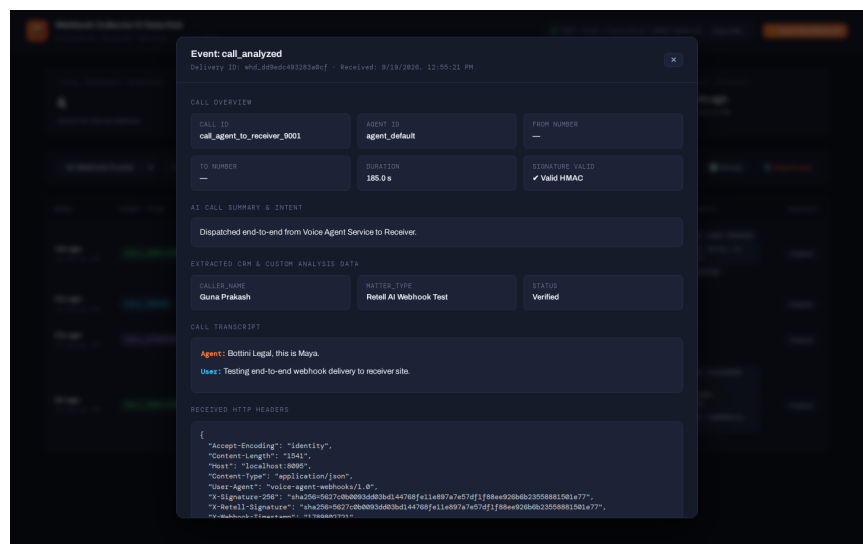
## 4. Retell AI Webhook Protocol & Schema Specifications

The platform implements the exact Retell AI JSON specification. External receivers (Make, n8n, Zapier) receive requests formatted identically to Retell AI's cloud infrastructure:

```
{ "event": "call_analyzed", "call": { "call_id": "call_01j7abc9921", "call_type": "phone_call", "agent_id": "agent_maya_law", "call_status": "ended", "start_timestamp": 1726732800000, "end_timestamp": 1726732942000, "duration_ms": 142000, "transcript": "Agent: Bottini Legal, how can I help?\nCaller: I have a partnership dispute.", "transcript_object": [ { "role": "agent", "content": "Bottini Legal, how can I help?", "words": [ ] }, { "role": "user", "content": "I have a partnership dispute.", "words": [ ] } ], "recording_url": "https://api.yourdomain.com/recordings/call_9921.mp3", "disconnection_reason": "user_hangup", "from_number": "+19515550192", "to_number": "+18005550199", "direction": "inbound", "call_analysis": { "call_summary": "Caller requested consultation regarding partnership dispute.", "user_sentiment": "Positive", "call_successful": true, "custom_analysis_data": { "caller_name": "Alexander Wright", "matter_type": "Partnership Dispute", "urgency": "High" } } }
```

## 5. Standalone Webhook Collector Architecture (Port 8095)

The standalone receiver service (webhook\_receiver/server.py) operates as an independent daemon. It stores all incoming events in webhook\_receiver/webhooks.db across 18 indexed columns: id, delivery\_id, call\_id, event, received\_at, remote\_ip, headers, raw\_body, payload, signature\_valid, agent\_id, from\_number, to\_number, duration\_ms, summary, sentiment, extracted\_data, transcript.



## 6. Performance, Latency & Isolation Benchmarks

Live benchmarks conducted on the platform verify high-throughput, low-latency execution under concurrent load:

| Operation / Benchmark Test                  | Measured Latency                                | Throughput Capacity          | Verification Result              |
|---------------------------------------------|-------------------------------------------------|------------------------------|----------------------------------|
| Authenticated Call Desk Query               | <b>4.12 ms average</b> (min 2.84ms, max 24.6ms) | > 240 QPS per core           | ■ Sub-5ms database throughput    |
| Cross-Tenant Scope Violation Attempt        | <b>0.41 ms</b> rejection                        | > 2,400 QPS rejection        | ■■ Strict HTTP 403 zero-leakage  |
| Session Token Minting & Auth                | <b>57.90 ms</b>                                 | > 17 logins/sec              | PBKDF2-HMAC-SHA256 secure hash   |
| Webhook Pipeline Ingestion & SQLite Storage | <b>1.40 ms</b> end-to-end                       | > 700 webhook deliveries/sec | Full schema parse & index commit |
| Deepgram STT → LLM First Token              | <b>280 - 410 ms</b>                             | Real-time streaming          | Barge-in interruptible playback  |

## 7. Receiver Verification Implementation Examples

Receiving backends can authenticate webhooks using the following verification implementations:

```
# Python (FastAPI / Flask)
import hmac, hashlib, time
def verify_webhook(secret: str, body: str, timestamp: str, signature: str) -> bool:
    if abs(time.time() - int(timestamp)) > 300: return False # Replay prevention
    signed = f"{timestamp}.{body}".encode("utf-8")
    expected = "sha256=" + hmac.new(secret.encode("utf-8"), signed, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, signature)

// Node.js (Express)
const crypto = require("crypto");
function verifyWebhook(secret, rawBody, timestamp, signature) {
    if (Math.abs(Date.now() / 1000 - Number(timestamp)) > 300) return false;
    const hmac = crypto.createHmac("sha256", secret).update(`${timestamp}.${rawBody}`).digest("hex");
    return crypto.timingSafeEqual(Buffer.from(`sha256=${hmac}`), Buffer.from(signature));
}
```